Лабораторная работа №3. Транспонирование матрицы и исследование локальности памяти

1. Система и условия измерений

Процессор: Intel Core i5-12450H, 12 логических процессоров (гибридный: 4 P- и 4 E-ядра). Кеши: L1d 320 КиБ (8 экземпляров), L2 7 МиБ (5 экземпляров), L3 12 МиБ (1 экземпляр). Компилятор: GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), ОС: Ubuntu 24.04. Вариант A: `double`, N = 3000. Сборка: `gcc transpose.c -O2 -fopenmp -o transpose`. Время — медиана 5 запусков после одного прогревочного (omp_get_wtime, без выделения и заполнения памяти); во всех запусках check = OK.

2. Результаты экспериментов

Эксперимент 1. Порядок обхода, 1 поток

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0,000191	0,000073	5,50	14,42
512	0,000837	0,000551	5,01	7,61
1024	0,005138	0,003987	3,27	4,21
2048	0,040875	0,037244	1,64	1,80
3000	0,030366	0,038118	4,74	3,78

Эксперимент 2. Блок, 1 поток, N = 3000

Блок b	Время, с	BW, ГБ/с
4	0,012687	11,35
8	0,012652	11,38
16	0,016471	8,74
32	0,019937	7,22
64 (b*)	**0,012035**	**11,97**
128	0,012459	11,56

Эксперимент 3. Потоки, N = 3000, b = b* = 64; T_1 = 0,029812 с (rows, 1 поток)

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0,031231	0,95	0,012197	2,44
2	0,015093	1,98	0,010773	2,77
4	0,010237	2,91	0,010290	2,90
8	0,013848	2,15	0,010052	2,97
16	0,012501	2,38	0,010560	2,82

Эксперимент 4. False sharing, 8 потоков, rows_omp

Стратегия	Время, с	BW, ГБ/с
schedule(static)	0,010979	13,12
schedule(static, 1)	0,013634	10,56
schedule(static, 8)	0,014521	9,92

3. Вывод

В transpose_rows чтение A идёт подряд, а запись в B с шагом 8N байт (в transpose_cols наоборот), поэтому в каждой версии один из потоков обращений на каждый элемент задевает новую кеш-линию (64 Б, из них используется 8 Б). Заметная разница получилась при N ≤ 1024 (cols быстрее в 2,6; 1,5 и 1,3 раза), при N = 2048 она около 10%, а при N = 3000 быстрее rows (в 1,26 раза), то есть устойчивого выигрыша у одной из версий нет; при этом две матрицы перестают помещаться в L3 (12 МиБ) уже при N = 1024 (по 8 МиБ каждая). Провал BW при N = 2048 (1,6–1,8 ГБ/с против 3,8–4,7 при N = 3000), вероятно, вызван тем, что шаг 16 КБ кратен степени двойки и элементы столбца попадают в одни и те же наборы кеша (конфликтные промахи). У блочной версии время почти не зависит от b при b = 4–8 и 64–128 (0,0120–0,0127 с), минимум при b* = 64; провалы при b = 16 и 32 (0,0165 и 0,0199 с) кешем не объясняются и, вероятно, являются шумом измерений. Пара блоков b×b занимает $2 \cdot b^2 \cdot 8$ Б, то есть 64 КБ при b* = 64: это больше L1d одного ядра (32–48 КиБ), но помещается в L2 (от 1,25 МиБ на ядро), а при b = 4–8 блоки малы, и накладные расходы на циклы по блокам дают примерно +5% к времени. Ускорение транспонирования ограничено сильнее, чем у интеграла (максимум 2,97 против 4,83 в ЛР2): вычислений нет, программа только читает и пишет по 72 МБ (намного больше L3 = 12 МиБ), поэтому упирается в пропускную способность памяти, общую для всех ядер, и время выходит на плато около 0,010 с (≈14 ГБ/с) уже при 2–4 потоках. У rows_omp при 8 и 16 потоках время даже хуже, чем при 4: процессор гибридный, и при равных статических кусках цикла, вероятно, все ждут самый медленный поток (на E-ядре), а 16 потоков больше, чем 12 аппаратных. При schedule(static, 1) соседние итерации i и i+1 достаются разным потокам, а в B они пишут в соседние элементы B[j·N+i] и B[j·N+i+1], то есть в одну кеш-линию: она постоянно переходит между ядрами (false sharing), и время растёт с 0,0110 до 0,0136 с (на 24%). При schedule(static) каждый поток пишет в непрерывный диапазон столбцов B, и общие линии бывают только на границах диапазонов. Чанк из 8 элементов не помог (0,0145 с), вероятно, потому что массив из malloc не выровнен по 64 Б (адрес % 64 = 16) и чанки не совпадают с границами кеш-линий.